

PROJECT REPORT.pdf



Shri Vile Parle Kelavani Mandal

Document Details

Submission ID

trn:oid::9832:137083488

Submission Date

Apr 29, 2026, 7:28 PM GMT+5:30

Download Date

Apr 29, 2026, 7:34 PM GMT+5:30

File Name

PROJECT REPORT.pdf

File Size

474.1 KB

42 Pages

12,601 Words

63,633 Characters

96% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups



70 AI-generated only 96%

Likely AI-generated text from a large-language model.



0 AI-generated text that was AI-paraphrased 0%

Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



PROJECT REPORT

On

Locus: A Gamified Mobile Learning App for Indian Competitive Exams

Submitted to

LOVELY PROFESSIONAL UNIVERSITY

In partial fulfilment of the requirements for the award of the degree of

Master of Computer Applications

Submitted By	Supervised By
Vinay Tiwari [REGISTRATION NO.]	[SUPERVISOR NAME] ([UID])
	[DESIGNATION]

LOVELY FACULTY OF TECHNOLOGY & SCIENCES

LOVELY PROFESSIONAL UNIVERSITY

PUNJAB

April 2026

CONTENTS OF THE REPORT

Section	Page No
Project Approval Format	(v)
Certificate	(vi)
Acknowledgement	(vii)
Abstract	(viii)
INTRODUCTION (THE PROBLEM)	1
CHAPTER 1: INTRODUCTION	2 – 6
1.1 About Locus	
1.2 The problem in plain words	
1.3 What I used to build it	
1.4 What I wanted the project to do	
CHAPTER 2: SYSTEM ANALYSIS	7 – 11
2.1 Why a tool like this is needed	
2.2 What's already out there vs. what Locus does	
2.3 Scope	
2.4 Feasibility	
CHAPTER 3: SYSTEM DESIGN	12 – 18
3.1 The big picture	
3.2 Database	

3.3 Backend modules	
3.4 The API surface	
CHAPTER 4: BACKEND IMPLEMENTATION	19 – 24
4.1 Bootstrap	
4.2 Auth	
4.3 The /me endpoints	
4.4 Courses	
4.5 Admin	
4.6 Code snippet	
CHAPTER 5: GAMIFICATION	25 – 31
5.1 The completion path	
5.2 Reward numbers	
5.3 Hearts	
5.4 Streaks	
5.5 Code snippet	
CHAPTER 6: LEADERBOARD	32 – 35
6.1 Tiers and groups	
6.2 The state machine	
6.3 Promotion and demotion	

6.4 Why Redis	
CHAPTER 7: MOBILE APP	36 – 40
7.1 Stack	
7.2 Screens	
7.3 State	
7.4 The exercise runner	
7.5 The HTTP client	
CHAPTER 8: ADMIN DASHBOARD AND CONTENT	41 – 43
8.1 The dashboard	
8.2 Privacy policy	
8.3 Seed pipeline	
8.4 Numbers	
CHAPTER 9: DEPLOYMENT	44 – 47
9.1 Containers	
9.2 Compose file	
9.3 The deploy script	
9.4 Production setup	
9.5 Building the AAB	
9.6 Security	

CHAPTER 10: TESTING AND RESULTS	48 – 52
10.1 How I tested	
10.2 Curl scripts	
10.3 The phone	
10.4 What actually shipped	
10.5 Sample API calls	
CHAPTER 11: WHAT'S BROKEN AND WHAT'S NEXT	53 – 56
11.1 Mistakes I made	
11.2 Limitations	
11.3 Future plans	
CHAPTER 12: CONCLUSION	57 – 58
CHAPTER 13: REFERENCES	59

PROJECT APPROVAL FORMAT

[To be inserted as per the university template — signed approval from the supervisor and the academic office.]

CERTIFICATE

I, **Vinay Tiwari ([REGISTRATION NO.]**), hereby declare that the work done by me on "**Locus: A Gamified Mobile Learning App for Indian Competitive Exams**" under **[SUPERVISOR NAME] ([UID])** from January 2026 to April 2026 at Lovely Professional University, Phagwara, Punjab, is a record of original work for the partial fulfilment of the requirements for the award of the degree **Master of Computer Applications**. The results

embodied in this report have not been submitted to any other subject or any other university for any award.

Date: [DATE]

Submitted By, Vinay Tiwari [REGISTRATION NO.]

Signature and Name of the Supervisor: [SUPERVISOR NAME] ([UID]) **Designation:** [DESIGNATION], School of Computer Applications **Date:**

Signature of AO: Date:

ACKNOWLEDGEMENT

I want to start by thanking my project guide, [SUPERVISOR NAME] ([UID]) of the School of Computer Applications, who put up with a lot of half-baked ideas from me over the last few months and never made me feel stupid for any of them. A bunch of the design decisions in this project look the way they do because of a single sentence or two from him in a review meeting.

I'm also thankful to **Dr. Lovi Raj Gupta**, Pro Vice Chancellor, and to the Head of the School of Computer Applications, for the kind of academic environment that lets a student actually try something this big as a capstone instead of being pushed toward a safe topic.

To my parents — thanks for not asking me what I was doing every time I disappeared for an entire weekend with my laptop. I know it must have looked weird from the outside.

And finally, thanks to the four or five friends who agreed to install the early Android builds on their phones, found the bugs that mattered, and sent me screenshots without being asked twice. The app is in much better shape because of them.

With due regards, **Vinay Tiwari** [REGISTRATION NO.]

Date: [DATE]

ABSTRACT

Locus is a mobile learning app I built for students preparing for India's big competitive exams. Right now it covers five of them: JEE Main, NEET, NIMCET, CUET, and UPSC Civil Services. The idea is pretty simple. Take the short, addictive lesson loop Duolingo uses for languages and bend it to fit an entrance-exam syllabus. You pick an exam, the app drops you into a feed of small exercises (each one takes about five minutes), and your progress is tracked through XP, hearts (basically your mistake budget), a daily streak, an in-app diamond currency, and a 30-tier league that moves you up or down every ten days based on how much XP you put in.

The project itself is four separate pieces that all live in one Git repo. The backend is a NestJS REST API talking to Postgres through Prisma. The phone app is React Native, built on top of Expo. There's also a small Next.js dashboard for managing content. The backend runs inside a Docker container on a 2GB DigitalOcean droplet in Bangalore. The leaderboard work is pushed out to Upstash Redis, because Redis sorted sets are basically built for ranking and trying to do the same thing in SQL would have been pretty miserable. The Android build is signed and sitting on the Google Play internal testing track.

At the time of writing, the seeded database has 5 exams, 24 subjects, 124 units, 465 exercises, and 4,650 questions. I wrote all of those by hand. There's a deterministic seed pipeline in the repo that loads them. The whole codebase is somewhere around 4,800 lines of TypeScript. The rest of this report goes through what got built, the calls I made and why, the things that broke along the way, and where I want to take the project from here.

INTRODUCTION (THE PROBLEM)

If you've ever watched a cousin or a friend prepare for JEE or NEET, the day looks familiar. A stack of textbooks. A folder of past-year papers. A Telegram group with thirty thousand silent people in it where somebody drops a PDF every few days. Two or three YouTube channels they actually trust. And somewhere on their phone, a coaching app they paid for in February and stopped opening by April. None of those things is bad on its own. The problem is that the student is the one stitching them together, and that stitching is real work, on top of the actual studying.

There's a quieter problem too, one nobody really wants to say out loud. A serious aspirant has maybe two or three hours of good attention in a day. That's it. And those hours don't show up as one clean block. They come in slivers: between school and tuition, before dinner, late at night when the house finally goes quiet. Almost nothing in this market is built for those slivers. Mock-test platforms want you to sit down for three uninterrupted hours. Live-class apps want you available at 7 PM sharp. Textbooks and PDFs are written like you'll finish a

whole chapter in one go. So the arc is always the same. Download. Two weeks of big enthusiasm. A slow drop off. Then a few months later the icon is just sitting there on the home screen, quietly making the student feel guilty.

Duolingo ran into a version of this exact problem in language learning about a decade ago, and the way they solved it was almost entirely behavioural rather than pedagogical. Lessons are short. Feedback is instant. A streak gives you something small to lose every day. A leaderboard adds a bit of harmless social pressure. None of that changes how a language is taught. But put together, it changes whether you open the app tomorrow evening.

The whole bet behind Locus is that the same loop works just as well for exam prep. The syllabus doesn't need rethinking. The questions don't either. What needs rethinking is the wrapper around them. Short sessions instead of long ones. Mechanics that actually make you want to come back the next day. That gap is what the project is trying to close.

CHAPTER 1: INTRODUCTION

1.1 About Locus

Locus is an Android app for students prepping for Indian competitive exams. Five of them, currently. JEE Main, NEET, NIMCET, CUET, and UPSC Civil Services. The content is laid out the obvious way. Each exam has subjects under it. Each subject has units. Each unit has exercises. Each exercise is around ten short questions.

The loop is supposed to be small. You open the app, you see a vertical feed of units in whatever subject you picked, you tap an exercise, and you're dropped into ten questions. When you finish, a results screen shows you the XP you just earned, how many hearts you have left, your current streak, and any diamond bonuses that fired. The whole thing is three to five minutes. That length is on purpose. If you close the app after one exercise, it should feel like you actually finished something, not like you gave up halfway.

Around the loop sits a small set of game mechanics. They're the part doing the retention work. **XP** is the lifetime counter, it only goes up. **Diamonds** are the in-app currency. You earn them at milestones (finishing a unit, hitting a 7-day streak) and you mostly spend them refilling hearts. **Hearts** are your mistake budget. You start with five. You lose one per wrong answer. You get one back every thirty minutes, or you pay 10 diamonds and refill instantly. **Streaks** count consecutive days you completed at least one exercise, in your local timezone. (Timezones turned out to be way more annoying than I expected, more on that later.) **Leagues** put you into an anonymous group of 12 at one of 30 tiers. Every ten days, top five move up, bottom three move down, and the whole thing resets.

The phone app is React Native and it talks to a NestJS REST API on top of Postgres. There's also a small Next.js dashboard so I can author content without going into the database directly. I built and shipped all four pieces by myself during the capstone period.

1.2 The problem in plain words

Three things kept bugging me when I looked at what was already on the market.

The first is practice volume versus what's actually possible. Every coaching pamphlet for JEE will tell you to solve thousands of problems before the exam. Fair. But almost nobody actually sits down and solves a hundred MCQs in one stretch, and the apps still pretend they do. So you end up with a tool asking for an hour at a time, and a student who has fifteen minutes between coaching classes. Something has to give and it's almost always the student.

The second is that there's no real reason to come back tomorrow. Most mock-test apps stop at the score screen. That's it. No streak you'd feel bad about breaking, no friend who's one position above you in a list, no notification at 8 PM saying "hey, you haven't practised today." These mechanics sound silly right up until you notice that Duolingo, Strava, and pretty much every fitness app on your phone all live or die by them. They aren't accidents.

And the third is that they run badly on cheap phones. A lot of the bigger Indian coaching apps are heavy hybrid wrappers. Long load times, choppy scrolling, an ad that plays before the actual content. The audience here isn't all on flagship phones. A real chunk of it is on 2GB Android devices and on those devices the experience is rough. A native, lightweight client is just a better deal for the people who need it the most.

Add those three up: nobody is shipping short sessions, a real engagement loop, and a lightweight client at the same time. Locus is my attempt at that combination.

1.3 What I used to build it

I'd rather explain why each piece is in here than just list the stack.

TypeScript is everywhere. Backend, mobile, admin, the seed scripts, all of it. The main reason is that I get to share types across project boundaries. A DTO defined on the backend can be imported by both the mobile API client and the admin form layer, so renaming one field surfaces compile errors in every place it's used. Which is mostly why it's there.

On the backend I went with NestJS 10.3.10. Every feature ends up in the same module-controller-service shape, so after the second or third one I stopped having to think about where to put a new file. The DI container does its job. You wire `PrismaService` in once and forget about it. After a couple of weeks the framework kind of stops being something you notice, which for a backend framework is about the best thing it can do.

The ORM is Prisma 5.18.0. The schema file is the single source of truth. Change a column, run `prisma generate`, the new types show up everywhere they should. Migrations are pretty easy too. `prisma migrate dev` in development, `prisma migrate deploy` when the container boots up in production.

The database is PostgreSQL 16. Mature, free, well-indexed, and there's an official Docker image that drops into Compose without any drama. JSONB exists, but I don't actually use it. Chapter 3 explains why.

For the leaderboard I'm using Redis through Upstash. The leaderboard wants sorted-set semantics with $O(\log N)$ updates and reads, which is basically what Redis was designed for. I could've put Redis on the same droplet but then I'd be on the hook for persistence, memory tuning and TLS, which is a lot of moving parts for what is basically a small cache. Upstash gives me a managed instance on the free tier, in the same Bangalore region as the API, so single-digit-millisecond round trips. Pretty easy call.

The phone app is Expo SDK 51 with React Native 0.74.5. The Expo managed workflow means I don't have to live inside Android Studio. EAS handles builds, OTA updates work out of the box, and I can produce a production AAB without ever opening Gradle. There's an exception to that when I have to patch a native library, but I get into that in Chapter 7.

Navigation is expo-router 3.5.23. File-system routing. Same idea as Next.js, which keeps the mental model the same between the mobile and admin folders. That's worth a lot when you're switching contexts five times in a day.

The admin dashboard is Next.js 14.2.15 with the App Router. Server components for reads, server actions for writes. Tailwind for styling, Radix UI for the accessible primitives, React Hook Form plus Zod for form state and validation. The neat thing about that combination is that the same Zod schema works as both the client-side validator and the shape I send to the backend, which kills off a whole class of "the form said one thing but the API expected another" bugs.

Deployment is Docker and Docker Compose. One multi-stage Dockerfile produces the production image, and a compose file wires it up to a Postgres container with a named volume. That's basically the whole deployment story. Hosting is DigitalOcean. A single 2GB droplet in BLR1. Cheap, simple, close to the users.

Auth is Passport JWT with bcryptjs. HS256, seven-day expiry, bcrypt at cost factor 10. Nothing clever, and I'd like to keep it that way. Auth code is one of those places where being a little bit dull is a feature.

1.4 What I wanted the project to do

The initial scope was kept small on purpose. A capstone is a fixed number of weeks. I'd rather ship one complete loop than half a dozen half-built features. Here's what I committed to.

A user has to register with email and password, log in next time, and stay logged in across app restarts. They have to either enrol in a full exam (with the exam date) or in individual subjects with a difficulty level. The content has to be browsable as Subject → Unit → Exercise → Question. Five question types: MCQ, MATCH, REORDER, COMPLETE, TRANSLATE. Finishing an exercise gives 20 XP. A perfect score adds another 5. Wrong answers cost hearts, capped at 5, regenerating one every 30 minutes, or refillable instantly for 10 diamonds each. There has to be a daily streak counter that respects the user's timezone and resets after a missed day. Every seventh day of an unbroken streak gives 150 diamonds. The first time a user finishes every exercise in a unit, they get 100. The leaderboard groups 12 users per cohort over a 10-day season. Top 5 promote, bottom 3 demote. An admin dashboard has to exist so that I (or anyone else) can author content without going near the database. And the whole backend has to actually be deployed on a server the mobile client can reach, with a signed AAB on the Google Play internal track.

Everything later in this report ties back to one of these.

CHAPTER 2: SYSTEM ANALYSIS

2.1 Why a tool like this is needed

Three things, taken together, made me think this was worth building.

The first was just watching the people around me. A friend buys a course, uses it religiously for about a week, then quietly stops opening the app. The content is usually fine. The issue is that the app expected an hour a day and they didn't have an hour a day. After a while the icon starts to feel like guilt rather than progress and it ends up in the third home-screen folder where apps go to die.

The second was paying attention to what actually makes Duolingo work. It isn't the language teaching. The teaching is fine, not extraordinary. What works is the streak. Luis von Ahn has said this almost in those words on a few occasions: the streak is the product. When I went looking through comparable Indian exam-prep apps, almost none of them had a streak. None had a real leaderboard. None had an in-app currency. The content was all there. The reason to come back tomorrow was not.

The third was a hardware test. I installed two of the bigger coaching apps on a 2GB Android phone (which is exactly the kind of device a lot of the target audience is on) and both took eight seconds or more to load, scrolled badly, and showed me an ad before letting me see anything educational. A native client that stays light on those phones is a real improvement, not a marketing line.

Add it up and the gap is clear. I don't need to invent new question formats or out-teach anyone on the syllabus. What I need to do is take normal exam content, chop it into short sessions, wrap a real engagement loop around it, and keep the binary small enough that a cheap phone doesn't choke on it.

2.2 What's already out there vs. what Locus does

The table below puts the typical design choices of the incumbent apps next to mine. I'm not trying to dunk on those apps. They're solving a different problem and they should be judged on that. But the contrast is useful to explain why Locus looks the way it does.

	Existing apps	Locus
Session length	One hour or longer	About 5 minutes per exercise
When you get feedback	At the end of a mock test	After every question
Engagement loop	Mostly nothing	XP, hearts, diamonds, streak, league
Content delivery	PDFs, video lectures, mock papers	Small interactive exercises
Question variety	Mostly MCQ	Five types (MCQ, MATCH, REORDER, COMPLETE, TRANSLATE)
Social pressure	Public leaderboard with thousands of people you'll never beat	Anonymous group of 12, you actually have a shot
Streak	None	Daily, in your timezone
Currency	Real money for premium content	Closed in-app diamond economy

Platform	Heavy hybrid web wrapper or video-first	Native Android via React Native
Admin	Black-box CMS or none	Self-hosted Next.js dashboard
Hosting	Cloud-only commercial	Self-hosted on a single small VPS

The first three rows of that table did most of the design work. Committing to five-minute sessions decides the shape of the unit feed, the structure of the exercise runner, the celebration screen, and even the hearts cap. Committing to a real engagement loop means the gamification subsystem has to be a first-class part of the codebase, not a sprinkle of badges on top. And committing to per-question feedback rules out a single end-of-exercise submission. The runner has to walk through questions one at a time and react to each.

2.3 Scope

What's in: email and password registration, login, and sessions that actually persist between app restarts. Enrolment, either by exam or by individual subjects. The Subject → Unit → Exercise → Question tree. All five question types. XP, diamonds, hearts, streaks, the whole league mechanic. An admin dashboard with full CRUD over the content. Deployment to a single cloud droplet. And a signed AAB sitting on the Google Play internal testing track.

What's out, and why. Real-money payments are off the table for the capstone. The diamond economy stays closed. The moment you take real money you also take on a payment gateway, KYC, refunds, and a stack of compliance work that has nothing to do with the academic side of this project, so I'd rather not. iOS is out for a more boring reason: EAS iOS builds need an Apple developer account, which costs money I didn't budget for. The Expo project itself is iOS-buildable in principle, I just haven't produced the binary. Push notifications are planned but deferred. The FCM setup is the kind of thing that deserves a careful pass rather than a rushed one. Multi-language content is out, the corpus is English-only, and translation is an editorial job not an engineering one. And video lectures or live tutoring are out because that's a completely different product with a different team and a different scope.

2.4 Feasibility

I looked at this in three ways. Technically nothing here is on the edge. JWT auth is standard. Sorted-set leaderboards are the textbook Redis use case. AABs from React Native projects are routine in 2026. The risks are about execution and time, not about whether the technology can actually do the job.

Economically the recurring cost is the droplet, which runs about \$6 a month, and Upstash on the free tier which is \$0. The Google Play developer account was a one-time \$25. So I can run the whole platform through the capstone for basically the price of a few coffees a month.

CHAPTER 3: SYSTEM DESIGN

The system is split into four sub-projects that share a single Git repo and only ever talk to each other over HTTP. The high-level topology is below.



-----+

A monorepo made sense because the database schema, the server contract, and the client types all have to stay in sync. Putting them in one repo turns runtime mismatches into compile-time errors, which I'll take every day of the week. The slightly heavier repo is a small price to pay, and TypeScript project references plus a sensible `.gitignore` keep the generated artefacts out of the way.

The four top-level folders do the following:

- **backend/** is the API. Business rules, database writes, JWT issuance, scheduled jobs. All of it lives here. From the phone app's perspective, "the platform" is one of the controllers in this folder.
- **mobile/** is the Android client, on Expo SDK 51, React Native 0.74.5, and expo-router. Almost all of the user-facing logic lives here.
- **admin/** is the Next.js 14 dashboard for content authoring. Single user, so the auth model is just a static admin token rather than full RBAC.
- **Courses/** holds the JSON corpus the seed pipeline reads from.

3.2 Database

The data model went through three iterations before it stopped changing on me.

Iteration 1. I was on SQLite to start with because it needed zero setup and I could commit the `.db` file to Git. Moving to Postgres later was painful, since constraint syntax and a few defaults are different. The lesson (pick your production database from day one) is the one I keep coming back to in Chapter 11.

Iteration 2. Each question type got its own table: `MCQQuestion`, `MatchQuestion`, `ReorderQuestion`, and so on, each with only the columns it actually needed. Looked clean for about ten minutes. The moment I added the fifth question type, every controller, every service and every API response had to dispatch on `type`, and the dispatch code was starting to outgrow the actual logic.

Iteration 3, where things stand now. One `Question` table. A `type` column. A flat `content` field that holds the type-specific payload as a JSON string. I gave up some structural neatness and got back about a third of the question-handling code.

3.2.1 The four data clusters

The schema breaks naturally into four mostly-independent clusters, which is convenient when something goes wrong. There's identity, which is just `User` and the bcrypt password hash. There's content, which is `Subject`, `Unit`, `Exercise`, `Question`, plus `Exam` and the

ExamSubject join table. There's onboarding (UserExamEnrollment and UserSubjectEnrollment) which captures what the user actually signed up to study. And then there's activity: UserStats, ExerciseAttempt, DiamondLedger, and the leaderboard pair LeaderboardGroup and LeaderboardParticipant. That last cluster is what the user is actually doing day to day.

Each cluster owns its own writes. The auth controller writes only to identity. Gamification writes only to activity. Admin writes only to content. Postgres isn't enforcing any of this for me, it's a discipline thing, but it has saved me real debugging time. When a counter looks wrong I already know which folder to open.

3.2.2 The content hierarchy in detail

A Subject row carries name, level (1, 2, or 3), and a unique slug. The level roughly maps to Class 10, Class 12, and undergrad work, so "Physics" shows up three times in the table at three levels. A composite uniqueness constraint on (name, level) keeps me from accidentally seeding "Physics" twice at the same level.

A Unit belongs to one Subject and has a title and an orderIndex. An Exercise belongs to a Unit and has a title, a points field that defaults to 20, and a duration field that defaults to 5 minutes. A Question belongs to an Exercise and carries a type from {MCQ, MATCH, REORDER, COMPLETE, TRANSLATE}, a prompt, a difficulty, a points value, and the content field above.

I store content as TEXT, not Postgres JSONB, for two reasons. One, the prototype was on SQLite, which has no JSONB, and a column-type migration was one less thing to deal with on the Postgres switch. Two, I validate the payload shape at the application boundary anyway: MCQ content needs an options array and a correct_index, MATCH content needs a map, and so on. So the speed of native JSON operators in the database doesn't buy me anything I'd actually notice.

3.2.3 Exam aggregation

Exam and Subject are linked through ExamSubject, which carries examId, subjectId, and orderIndex. A single subject row can therefore appear in multiple exams without duplication. "Mathematics" at level 2, for example, is shared across JEE Main, NIMCET, CUET, and UPSC. Adding a new exam comes down to inserting the right join rows.

3.2.4 Activity ledger

Every completed exercise produces an ExerciseAttempt row with correctCount, totalCount, scorePct, xpAwarded, diamondsAwarded, isPerfect, and a timestamp.

This row is the canonical record of a session. The home-screen completion set, any future analytics, and the planned mastery calculations are all derived from it.

Diamond movements live in their own DiamondLedger table. Each row has `userId`, a signed `delta`, a reason from a small enum (`UNIT_COMPLETION`, `STREAK_7`, `HEART_REFILL`, `MANUAL`), and an optional `refId`. The invariant is that the sum of `delta` per user equals the `diamonds` field on `UserStats`. I have a small reconciliation script that checks this against production. It has been clean every time I've run it.

3.2.5 The schema as a table

Model	Important fields
User	<code>id</code> , <code>email</code> (unique), <code>passwordHash</code> , <code>displayName</code> , <code>timezone</code>
Subject	<code>id</code> , <code>slug</code> (unique), <code>name</code> , <code>level</code> , <code>displayName</code> , (<code>name</code> , <code>level</code>) unique
Unit	<code>id</code> , <code>subjectId</code> , <code>title</code> , <code>orderIndex</code>
Exercise	<code>id</code> , <code>unitId</code> , <code>title</code> , <code>points</code> , <code>duration</code>
Question	<code>id</code> , <code>exerciseId</code> , <code>type</code> , <code>prompt</code> , <code>difficulty</code> , <code>points</code> , <code>content</code>
Exam	<code>id</code> , <code>slug</code> , <code>title</code>
ExamSubject	(<code>examId</code> , <code>subjectId</code>) PK, <code>orderIndex</code>
UserExamEnrollment	<code>userId</code> , <code>examId</code> , <code>examDate</code>
UserSubjectEnrollment	<code>userId</code> , <code>subjectId</code> , <code>difficulty</code>
UserStats	<code>userId</code> (PK), <code>totalXp</code> , <code>diamonds</code> , <code>hearts</code> , <code>lastHeartRefillAt</code> , <code>currentStreak</code> , <code>longestStreak</code> , <code>currentLeagueIndex</code> , <code>lastActivityDate</code>
ExerciseAttempt	<code>id</code> , <code>userId</code> , <code>exerciseId</code> , <code>correctCount</code> , <code>totalCount</code> , <code>scorePct</code> , <code>xpAwarded</code> , <code>diamondsAwarded</code> , <code>isPerfect</code>
DiamondLedger	<code>id</code> , <code>userId</code> , <code>delta</code> , <code>reason</code> , <code>refId</code>

LeaderboardGroup	id, leagueTier, state, capacity, filledCount, startedAt, endsAt
LeaderboardParticipant	groupId, userId, seasonXp, joinedAt, lastXpAt

The full schema source is available at [backend/prisma/schema.prisma](#).

3.2.6 Indexing

I added indexes wherever a list query needed to stay snappy as the data grew:

- A composite index on (userId, createdAt) for ExerciseAttempt, for the timeline view I want to add later.
- A composite index on (leagueTier, state) for LeaderboardGroup. The "find an open group at this tier" query runs on every XP-earning event, so this one is hot, and I don't want it doing a sequential scan.
- An index on Question.exerciseId, since I always filter on it when pulling the questions for an exercise.

All of these are declared in schema.prisma and live in the migration history.

3.3 Backend modules

The backend is broken up into eleven NestJS modules. Each owns one feature.

Module	Responsibility
ConfigModule	Reads .env and exposes a typed ConfigService
ScheduleModule	Registers cron jobs
PrismaModule	Provides a single global PrismaClient
RedisModule	Provides a single global ioredis client; degrades gracefully when REDIS_URL is unset
AuthModule	Registration, login, JWT issuance, and JWT verification
UsersModule	Current user, enrolments, and stats
CoursesModule	Read-only views over content
AdminModule	Full CRUD over content, gated by the static admin token

GamificationModule	Exercise completion and reward processing
LeaderboardModule	League state, group membership, and ranking
JobsModule	The midnight streak-reset cron

3.4 The API surface

Everything is mounted under `/api`. The semantics are the usual REST ones. GET to read, POST for creation and side-effects, PUT to update, DELETE to remove, JSON in and out, standard HTTP status codes.

User-facing endpoints:

Method	Path	Description
POST	<code>/api/auth/register</code>	Create a user
POST	<code>/api/auth/login</code>	Log in
GET	<code>/api/me</code>	Current user with enrolments and stats
GET	<code>/api/me/stats</code>	Live stats with regenerated hearts applied
POST	<code>/api/me/enrollments/exam</code>	Set the exam enrolment
POST	<code>/api/me/enrollments/subjects</code>	Set the subject enrolments
POST	<code>/api/me/hearts/refill</code>	Spend diamonds to refill hearts
GET	<code>/api/courses/exams</code>	All exams with their subjects
GET	<code>/api/courses/subjects</code>	All subjects, optionally filtered by level
GET	<code>/api/courses/subjects/:id</code>	A single subject with units and exercises
GET	<code>/api/courses/units/:id</code>	A single unit with its exercises
GET	<code>/api/courses/exercises/:id</code>	A single exercise with all its questions
POST	<code>/api/exercise/complete</code>	Submit a completed exercise

GET	/api/leaderboard/me	Current group, current rank, and season countdown
POST	/api/leaderboard/join	Force a join (boot path)

Administrative endpoints mirror the content tables with full CRUD and are gated by the `x-admin-token` header.

CHAPTER 4: BACKEND IMPLEMENTATION

4.1 Bootstrap

Bootstrap lives in `backend/src/main.ts`. Three things happen before the server starts listening on port 3000. A global prefix of `/api` is set, so every controller route gets namespaced automatically. The `ValidationPipe` is registered with `whitelist: true` and `transform: true`, which silently drops unknown fields from incoming request bodies and coerces what's left to the types declared on the DTOs. CORS is enabled so the mobile app can hit the server during local development and the admin dashboard can hit it everywhere.

4.2 Auth

`AuthService` exposes two public methods, `register` and `login`, both returning the same envelope (a user summary plus a signed JWT) through a private `signAndShape` helper. Passwords are hashed with `bcrypt` at cost factor 10. The JWT payload only carries `sub` (the user ID) and `email`. The signing secret comes from `JWT_SECRET` and the expiry from `JWT_EXPIRES_IN`, set to seven days in production.

Verification is done by `JwtStrategy`. It pulls the bearer token from the `Authorization` header, checks the signature, and loads the matching `User` row from the database. A `@CurrentUser()` parameter decorator lets controllers grab the authenticated user straight from their parameter list without any boilerplate. Any route that needs auth declares `@UseGuards(JwtAuthGuard)`.

4.3 The /me endpoints

`UsersController` covers the small handful of endpoints centred on the current user. The interesting one is `GET /me/stats`. Instead of just returning the row from the database, the handler loads it, applies the heart-regeneration calculation from Chapter 5, persists the corrected count, and returns it together with the timestamp at which the next heart will be available.

The point of doing it this way is to avoid the obvious-but-wrong design where the mobile client polls `/me/stats` every minute to keep the heart count current. With the next-regen timestamp returned once at app start, the client can run its own countdown locally and only re-query when the user actually does something that changes state.

4.4 Courses

`CoursesController` exposes five read-only endpoints. The exercise endpoint is the only one that does anything beyond a direct Prisma query. Every question returned from `/exercises/:id` has its `content` string passed through `JSON.parse` before serialisation, so the client gets a structured object and never has to know about the underlying storage shape. If a parse fails (it hasn't happened in production, but I deliberately broke a row once to see what would happen) the question comes back with `content: null` and a warning gets logged. One bad row doesn't take the whole exercise down with it.

4.5 Admin

`AdminModule` mirrors the content tree with full CRUD over subjects, exams, units, exercises, and questions. List views support pagination and a basic title search. Every admin route is behind `AdminGuard`, which compares the `x-admin-token` request header against the `ADMIN_API_TOKEN` environment variable. The token is a 32-byte base64 string from `openssl rand -base64 32`. Not something anyone is guessing.

The `POST /admin/questions` and `PUT /admin/questions/:id` endpoints take the question payload as an object in the request body and run it through `JSON.stringify` before handing it to Prisma. The read path does the inverse. So the dashboard never deals with JSON serialisation directly, and the storage shape stays uniform.

4.6 Code snippet

Here is the auth service, lightly trimmed.

```
// backend/src/auth/auth.service.ts (excerpt)
@Injectable()
export class AuthService {
  constructor(
    private readonly prisma: PrismaService,
    private readonly jwt: JwtService,
  ) {}

  async register(dto: RegisterDto) {
    const passwordHash = await bcrypt.hash(dto.password, 10);
    const user = await this.prisma.user.create({
```

```
    data: {
      email: dto.email,
      passwordHash,
      displayName: dto.displayName,
      timezone: dto.timezone ?? 'UTC',
      stats: { create: { hearts: MAX_HEARTS } },
    },
  });
  return this.signAndShape(user);
}

async login(dto: LoginDto) {
  const user = await this.prisma.user.findUnique({
    where: { email: dto.email },
  });
  if (!user) throw new UnauthorizedException();
  const ok = await bcrypt.compare(dto.password,
  user.passwordHash);
  if (!ok) throw new UnauthorizedException();
  return this.signAndShape(user);
}

private signAndShape(user: User) {
  const token = this.jwt.sign({ sub: user.id, email:
  user.email });
  return {
    user: { id: user.id, email: user.email, displayName:
  user.displayName },
    token,
  };
}
```

An auth service should be predictable, not clever. This one is.

CHAPTER 5: GAMIFICATION

The gamification subsystem is where most of the actual logic in the platform lives. Almost every state transition the user notices passes through one method, `GamificationService.completeExercise`. Funnelling everything through one entry

point was an early call I stuck with all the way through. The win is that the state machine stays traceable, the audit trail is easy to read, and there's one surface for tests to hit. The cost is that the method itself is long. It stays readable because each step is a short, self-contained block.

5.1 The completion path

Here's roughly what happens when an exercise gets submitted. The phone posts an array of booleans, one per question, to `POST /api/exercise/complete` with the exercise ID. The service opens one Prisma transaction and works through the request inside it.

First it loads the exercise with its questions. If the boolean array length doesn't match the question count it throws `BadRequestException` and bails — I don't try to be clever and partially process a malformed body. Then it walks the array to get `correctCount` and `totalCount`.

Next it deals with hearts. One heart per wrong answer, never going below zero. If the user came in with zero hearts already, the request gets rejected with `ForbiddenException` and a body of `{ "code": "OUT_OF_HEARTS" }` so the phone can pop the right modal without inferring it from the status code. After hearts, XP: 20 base, plus 5 if every answer was right. Both go onto `UserStats.totalXp` along with a fresh `lastActivityDate`.

Then the streak. `computeStreak` (covered in 5.4) returns one of `SAME_DAY`, `CONTINUED`, `RESET`, `NEW`, and `currentStreak` gets updated to match. `longestStreak` only moves on `CONTINUED` or `NEW`. That sounds like a footnote, but an early version of this let `longestStreak` go *down*, and the bug it caused took an embarrassingly long time to find.

After that there's bookkeeping. An `ExerciseAttempt` row is inserted with the inputs, the outputs, and the timing — basically the full record of the session. If every exercise in the parent unit has been finished at least once, the user gets 100 diamonds (reason `UNIT_COMPLETION`) and a matching ledger row. If `currentStreak % 7 === 0`, another 150 (reason `STREAK_7`). Last thing in the transaction is handing the XP delta to `LeaderboardService.recordXp`, so the rank in the active group catches up immediately.

The method returns a `CompleteExerciseResult` carrying score percentage, perfect-score flag, XP awarded, diamonds awarded, hearts left, streak outcome, new streak length, and the unit-completion flag. One response, everything the celebration screen needs, no follow-up requests.

5.2 Reward numbers

All the reward numbers live in `backend/src/gamification/rewards.constants.ts` so I don't have to grep through the codebase if I ever want to change one.

Constant	Value	Meaning
EXERCISE_BASE_XP	20	XP awarded for completing any exercise
PERFECT_SCORE_BONUS_XP	5	Additional XP for a perfect score
UNIT_COMPLETION_DIAMONDS	100	Diamonds awarded the first time every exercise in a unit is completed
STREAK_7_DIAMONDS	150	Diamonds awarded when <code>currentStreak % 7 === 0</code>
MAX_HEARTS	5	Hard cap on hearts
HEART_REGEN_INTERVAL_MS	1,800,000	30 minutes per regenerated heart
HEART_REFILL_COST_DIAMONDS	10	Diamond cost per heart purchased

These numbers roughly track Duolingo's published settings. I haven't run any controlled tuning on them. Calibration is on the future-work list.

5.3 Hearts

Heart logic lives in `backend/src/gamification/hearts.util.ts` and is written as pure functions. Each one takes the previous state and returns the next state. None of them touches the database or calls `new Date()` directly. The current time gets passed in by an adapter inside the service, which is also the thing responsible for persisting the result.

The reason for being strict about purity here is that the same calculation has to run in two places: on the server when processing a completion, and on the client when rendering the next-heart countdown. Pure functions are the most reliable way to make sure both sides agree on what the heart count should be at any given moment.

Two functions do all the work.

`applyHeartRegen(stored, lastRefillAt, now)` returns the heart count that should currently apply. If the stored value is already at the cap, it returns the cap and a null next-refill timestamp. Otherwise, it counts how many complete 30-minute windows have gone by since `lastRefillAt`, adds that to the stored value, clips to the cap, and returns both the new count and the timestamp at which the next regen tick will happen. The phone uses that timestamp to drive its on-screen countdown.

`loseHearts(state, delta)` deducts hearts and (this is the important bit) kicks off the regen timer the first time the user drops below the cap. If the user is already below the cap

and the timer is already running, it leaves it alone. Restarting the timer on every loss would unfairly stretch out the wait, which was a bug in an early build that an external tester reported within an hour of installing.

5.4 Streaks

Streak logic lives in `backend/src/gamification/streak.util.ts` and respects the user's local timezone. The `User.timezone` field defaults to "UTC" if no value is supplied. The whole computation comes down to a small decision tree.

Condition	Outcome	Effect
<code>lastActivityDate</code> is null	NEW	<code>currentStreak = 1</code>
Last activity occurred today	SAME_DAY	unchanged
Last activity occurred yesterday	CONTINUED	<code>currentStreak += 1</code>
Otherwise	RESET	<code>currentStreak = 1</code>

The "today" and "yesterday" comparisons are done against YYYY-MM-DD keys derived in the user's timezone, not in UTC. So an exercise finished at 1 AM local time in Asia/Kolkata is correctly counted on the new day, not the previous one.

A scheduled job in `JobsModule` walks the `User` table once per day at 12:01 UTC. For any user whose `lastActivityDate` is older than yesterday in their own timezone and whose `currentStreak` is positive, the streak gets reset to zero. Without this job, a user coming back after a long absence would see a stale streak on the home screen, which is confusing and wrong. With it, the streak you see on screen is always the streak you actually have.

5.5 Code snippet

The streak utility, lightly trimmed:

```
// backend/src/gamification/streak.util.ts (excerpt)
export type StreakOutcome = 'NEW' | 'SAME_DAY' | 'CONTINUED' |
'RESET';

export function computeStreak(
  prev: { lastActivityDate: Date | null; currentStreak: number;
  longestStreak: number },
  now: Date,
  timezone: string,
): { outcome: StreakOutcome; currentStreak: number; longestStreak:
```

```
number } {  
  if (!prev.lastActivityDate) {  
    return {  
      outcome: 'NEW',  
      currentStreak: 1,  
      longestStreak: Math.max(1, prev.longestStreak),  
    };  
  }  
  const todayKey = dayKey(now, timezone);  
  const lastKey = dayKey(prev.lastActivityDate, timezone);  
  const yesterdayKey = dayKey(addDays(now, -1), timezone);  
  
  if (lastKey === todayKey) {  
    return {  
      outcome: 'SAME_DAY',  
      currentStreak: prev.currentStreak,  
      longestStreak: prev.longestStreak,  
    };  
  }  
  if (lastKey === yesterdayKey) {  
    const next = prev.currentStreak + 1;  
    return {  
      outcome: 'CONTINUED',  
      currentStreak: next,  
      longestStreak: Math.max(next, prev.longestStreak),  
    };  
  }  
  return {  
    outcome: 'RESET',  
    currentStreak: 1,  
    longestStreak: prev.longestStreak,  
  };  
}
```

CHAPTER 6: LEADERBOARD

The leaderboard is the social half of the engagement loop. Two hard constraints shaped its design. First, the user always has to be in some group, because an empty leaderboard screen gives them nowhere to go. Second, updates have to be cheap, because they fire on every completed exercise, which is the most frequent write in the system.

6.1 Tiers and groups

There are thirty tiers. New users start at tier 1 and can climb to tier 30 by sticking with it. Each tier has a colour, a name (Bronze, Silver, Gold, all the way up to Diamond and Obsidian), and a set of badge icons. Tier 1 is sticky at the floor, you can't demote below it, and tier 30 is sticky at the ceiling.

Each group caps at twelve participants. The number is borrowed from Duolingo's earlier Diamond league sizing. Twelve is small enough that the whole roster fits on one screen with no scrolling, and big enough to justify a meaningful split into top five, middle four, bottom three.

6.2 The state machine

A `LeaderboardGroup` is in one of three states. `WAITING` is when there are fewer than twelve participants in it, new joiners get appended, and season XP isn't being accumulated yet. `ACTIVE` is when the group has exactly twelve participants, the ten-day timer is running, and each XP delta from a completed exercise gets added to the participant's `seasonXp`. `CLOSED` is when the season timer has expired, ranking is frozen, and promotions and demotions are applied.

When a user needs a group (either by hitting `POST /api/leaderboard/join` directly or implicitly on the first XP-earning event of the season) the service goes looking for a `WAITING` group at the user's `currentLeagueIndex`. If one exists, the user gets dropped into it. If not, a new one gets created. The transition from `WAITING` to `ACTIVE` happens the moment a join brings `filledCount` to twelve. At that point `startedAt` becomes the current timestamp and `endsAt` gets set ten days out.

6.3 Promotion and demotion

When a group's timer expires, participants get sorted by `seasonXp` descending, with `lastXpAt` ascending as the tiebreaker so an earlier completion beats a later one at equal XP. Top five promote one tier, middle four stay put, bottom three demote one tier. Tier 1 is sticky at the floor, so a demotion at tier 1 is a no-op. Same for tier 30 at the top.

The user's `currentLeagueIndex` on `UserStats` is updated, the group is marked `CLOSED`, and the join algorithm runs again at the new tier on the next XP-earning request. Closure is lazy. There's no separate scheduler running this. The next request does it.

6.4 Why Redis

A SQL-based leaderboard would, in principle, be fine. A query like `SELECT ... ORDER BY season_xp DESC LIMIT 12` against `LeaderboardParticipant` would do the job.

The trouble is that the sort runs on every read while the `season_xp` column is updated on every write, and both sides scale badly under concurrent load.

Redis sorted sets are made for exactly this access pattern. One sorted set per group, keyed `leaderboard:{groupId}`. Members are user IDs, scores are `seasonXp` values. Updates use `ZINCRBY`, which is $O(\log N)$. Reads use `ZREVRANGE WITHSCORES` for the top-twelve list and `ZREVRANK` for the user's rank, both also $O(\log N)$. With twelve members per set, latency on Upstash sits below a millisecond, consistently.

The Redis client in `backend/src/redis/redis.module.ts` is wrapped in a factory that returns `null` when `REDIS_URL` is unset. The leaderboard service notices that and falls back to a SQL query. So a fresh checkout of the repo runs the entire backend on a developer machine without needing Redis at all. The latency profile changes. The behaviour doesn't.

CHAPTER 7: MOBILE APP

7.1 Stack

The phone app is built on Expo SDK 51, React Native 0.74.5, and expo-router 3.5.23. The visual theme is dark, with a near-black background and a green accent that consciously borrows from Duolingo. Theme tokens live in `mobile/src/theme/colors.ts`. Local persistent state is handled with `@react-native-async-storage/async-storage`. The Android build targets API level 35, which Google Play has required for new submissions since August 2025.

7.2 Screens

In expo-router, the file system is the navigation graph. Wherever you put a file under `mobile/app/` is what determines the route. Here's what shipped.

Route file	Purpose
<code>mobile/app/_layout.tsx</code>	Root layout; wraps the navigator in the three context providers
<code>mobile/app/index.tsx</code>	Boot router; redirects to <code>/auth</code> , <code>/onboarding</code> , or <code>/(tabs)/home</code> based on state
<code>mobile/app/auth.tsx</code>	Combined login and registration form
<code>mobile/app/onboarding.tsx</code>	Exam selection (with date) or subject selection

mobile/app/(tabs)/home.tsx	Primary feed of units and exercises
mobile/app/(tabs)/leaderboard.tsx	Current group, current rank, and season countdown
mobile/app/(tabs)/profile.tsx	Identity, sign-out, and settings
mobile/app/exercise/[id].tsx	Question runner
mobile/app/streak.tsx	Streak detail screen
mobile/app/shop.tsx	Heart refill page

7.3 State

Client state is split across three React contexts in [mobile/src/context/](#). `AuthContext` holds the current user, the JWT, the enrolment record, and a `ready` flag that the boot router waits on before rendering anything at all. `ProgressContext` holds the gamified state (`totalXp`, `diamonds`, `hearts`, `currentStreak`, `currentLeagueIndex`, `nextHeartAt`) and the two actions `refreshStats` and `refillHeart`. `CourseContext` caches the lists of exams, subjects, units and exercises so that navigation within a session doesn't keep hammering the network for the same data over and over.

I deliberately didn't pull in Redux or Zustand. The state is small, producers and consumers stay localised, and the React DevTools profiler doesn't show any rerender storms under normal use. Adding a state library would buy bundle size without buying me anything I actually need. If the app gets ten times bigger I'll revisit it, but not before.

7.4 The exercise runner

The runner is the most important screen in the app. It walks through the questions one by one, renders the right widget for each, builds up a boolean array of correct-or-incorrect outcomes, and posts the result to the server when the last question is answered.

The runner itself stays short because every question type is dispatched through a tiny renderer, [QuestionRenderer.tsx](#). It looks at `question.type` and routes to one of the components below.

Component	Type	Behaviour
<code>MCQ.tsx</code>	MCQ	Vertical list of options; locks the choice on tap and marks it correct or incorrect
<code>Match.tsx</code>	MATCH	Two columns; pairs are formed by tapping a left item followed by a right item

Reorder.tsx	REORDER	Long-press-and-drag list
Complete.tsx	COMPLETE	A sentence containing a blank, paired with a single text input
Translate.tsx	TRANSLATE	Source text accompanied by a free-text input

Each renderer reports its result through `onAnswered(correct: boolean)`. The runner collects those booleans and posts them to `POST /api/exercise/complete` once the last question is answered. The response then drives the celebration screen, which animates the XP, the diamonds, the new heart count, and any streak milestones in sequence.

7.5 The HTTP client

`mobile/src/api/client.ts` is deliberately tiny. One generic `request<T>` function does the fetch, attaches `Authorization: Bearer <jwt>` if a token is present, throws a typed error on a non-2xx response, and returns `await res.json()` on success. Every public method on the `api` object (`api.login`, `api.completeExercise`, `api.fetchSubject`, and so on) is a one-liner around `request`. The base URL comes from `EXPO_PUBLIC_API_URL`, which is `http://localhost:3001/api` in development and the production droplet URL in release builds.

There's a small components folder for the reusable building blocks: `DuoButton` (the green pill CTA), `LessonNode` (the circular node in the unit feed), `OutOfHearts` (the modal you see when you try to start an exercise with zero hearts), `ProgressBar`, `SubjectIcon`, `TopNav`, and `UnitHeader`. The set is deliberately conventional. There's no design-system overhead here.

7.5.1 Two bugs worth recording

Two bugs from the release stretch deserve to be written down, because they ate a lot of time and they're exactly the kind of thing that only shows up once you actually deploy.

The first was a Kotlin `removeLast()` crash on Android 15. The `react-native-screens` library calls `MutableList.removeLast()` during navigation pop. Android 15 changed the behaviour of that method so that calling it on an empty list throws `UnsupportedOperationException`. The fix is a one-line patch via `patch-package`, replacing `removeLast()` with `removeAt(list.lastIndex)`. I didn't catch this locally. The Google Play pre-launch report did.

The second was a null-array bug in `expo-modules-core PermissionsService`. This one only fires when `compileSdk` is bumped to 35, which Google Play forces on you. Same fix

shape, a one-line patch. Both patches live in `mobile/patches/` and get reapplied automatically through a `postinstall` script.

CHAPTER 8: ADMIN DASHBOARD AND CONTENT

8.1 The dashboard

The admin dashboard is a small Next.js 14 app in `admin/`. App Router throughout, Tailwind for styling, Radix UI for accessible primitives, React Hook Form with Zod for forms. There's one user (me), so the auth model collapses down to a static admin token.

There is a screen for each content table: Subjects, Exams, Units, Exercises, Questions. Each one with a paginated list view (with title search), a creation form, and an edit form. Every form is driven by a Zod schema that mirrors the matching NestJS DTO, so a shape mismatch on the client gets caught before the request goes out. There's also a small overview screen with row counts across the five tables, which is what I look at first after every deploy as a sanity check.

The fetch wrapper in `admin/src/lib/admin-api.ts` attaches the `x-admin-token` header automatically on the server side. So the token never reaches the browser. Every mutation flows through Next.js server actions or route handlers, with the token injected from the server environment.

8.2 Privacy policy

Google Play requires a publicly hosted privacy-policy URL before they will let you submit. Rather than spinning up a separate static site, I serve the policy from the admin app at `/privacy`. It's a server component and includes the effective date, a contact email, and the standard sections on data collection, usage, retention, deletion, user rights, and contact info. Saves me a domain and a hosting plan.

8.3 Seed pipeline

The seed pipeline turns an empty Postgres database into a usable app. It lives in `backend/prisma/` and is structured in three layers, although they're not really layers in any clean architectural sense, they're just three files with three jobs.

The lower one is the per-subject question banks in `backend/prisma/seed-data/`, files like `cpp.ts`, `dsa.ts`, `python.ts`. Each file exports a typed list of question records, each one carrying the prompt text, the answer payload in the JSON shape the runtime expects, and an explanation. These I wrote by hand. I did try generated alternatives at one point but the wording quality wasn't consistently good enough to ship.

The middle one does assembly. `backend/prisma/seed-bank.ts` glues the per-subject banks into a `(subjectName, level) → Unit[]` mapping. It owns the slug generator, the unit ordering, and the exercise grouping (about ten questions per exercise). It's the only file that needs to know the canonical spelling of each subject.

The top one is orchestration. `backend/prisma/seed.ts` is the entry point. It runs the steps in dependency order, which is: clear existing rows, insert subjects across the three levels, insert units, insert exercises, insert questions, link subjects to exams via `ExamSubject`. The full pipeline runs in about seven seconds against a local Postgres on a typical laptop, and around twelve on the smaller production droplet.

The seed runs through `npm run seed`. The deploy script triggers it automatically on a fresh droplet whenever the `Subject` table is empty.

8.4 Numbers

After the seed runs, the database has these row counts.

Table	Rows
Exam	5
Subject	24
Unit	124
Exercise	465
Question	4,650

4,650 questions is small compared to the practice volume a serious aspirant is usually told to grind through, which is typically tens of thousands. Growing the corpus is a content problem, not an engineering one. The schema scales without any change. The `Question` table will sit comfortably with a million rows on the same droplet. Corpus growth is on the future-work list.

CHAPTER 9: DEPLOYMENT

A capstone that lives only on a developer laptop isn't really finished. So Locus runs on a public DigitalOcean droplet, the phone app talks to it over the open internet, and deployment is one bash invocation.

9.1 Containers

The backend gets containerised through a multi-stage Dockerfile, [backend/Dockerfile](#). The build stage installs dependencies, runs `prisma generate`, compiles TypeScript via `nest build`, and prunes dev dependencies. The runtime stage starts fresh from `node:20-alpine`, copies in the compiled output, the pruned `node_modules`, and the Prisma schema, then installs [ts-node@10.9.2](#) and [typescript@5.5.4](#) globally so that the seed script can run inside the container, and exposes port 3000.

The default CMD is `npx prisma migrate deploy && node dist/src/main.js`, so each container start applies any pending migrations before booting the API. Net effect: deployment is `docker compose up -d --build`, with migrations handled on container start, no manual step in between.

9.2 Compose file

[backend/docker-compose.yml](#) wires the API container to a `postgres:16-alpine` container backed by a named volume for the data directory, and binds the API to `127.0.0.1:3001` on the host so a reverse proxy on the droplet can route to it. Redis is intentionally not in the compose file, because production uses Upstash, accessed over TLS via `REDIS_URL`. Postgres credentials come in through environment variables.

9.3 The deploy script

[backend/deploy.sh](#) is a small idempotent bash script. It pulls the Postgres image, builds the API image, starts the database, waits for `pg_isready`, starts the API (which handles migrations itself), and runs the seed only if the `Subject` table is empty. The seed check is a tiny `psql` query so it doesn't re-run on subsequent deploys.

After that, every deployment is one line:

```
cd ~/Locus/backend && git pull && ./deploy.sh
```

This stays true even when the schema has changed, because migrations apply on container start.

9.4 Production setup

Production is a single 2GB DigitalOcean droplet running Ubuntu 24.04 in BLR1, with public IP `144.126.255.209`. The API is exposed on port 3001. Redis is on the Upstash free tier, also in BLR1, which gets me single-digit-millisecond round trips. Total monthly recurring cost is about \$6 for the droplet, \$0 for Redis on the free tier.

9.5 Building the AAB

The signed Android App Bundle was built locally with the standard Gradle pipeline (`./gradlew bundleRelease`) and signed with a project-specific RSA-2048 keystore valid for 10,000 days. The keystore file and password live on the build machine and don't get committed.

The AAB went up to the Google Play Console and onto the internal testing track. Two pre-launch report failures happened (those are the two patches from Chapter 7) and the third upload passed cleanly and got promoted. The version code went from 1 to 4 across those iterations. The version name stayed at 1.0.0 because none of the user-visible behaviour had changed.

9.6 Security

Security on something this small is really about getting the basic primitives right and resisting the urge to invent new ones. Passwords go through `bcrypt`, cost factor 10. The hash isn't returned by any endpoint and is only read in one place, `AuthService.login`, for the comparison. Protected routes sit behind `JwtAuthGuard`. JWTs are HS256 with a 64-char secret out of `openssl rand -hex 32`. The secret can be rotated without code changes — I've already had to do that once.

The admin side uses a static `x-admin-token` header with 32 bytes of base64 entropy. Not realistically guessable. Full RBAC was skipped because it'd be silly for a one-user dashboard. DTOs run through `class-validator` with `whitelist: true`, which strips unknown fields before they reach a service. Mass assignment basically can't happen as a result. All database access is via Prisma, all queries parameterised. No string-concatenated SQL anywhere. Secrets are in `.env` files on the droplet, gitignored, with a `.env.example` checked in so the required keys are at least documented.

The big outstanding gap is TLS. The API is currently serving the phone app over plain HTTP. The next iteration is going to put Caddy in front of it and move the client to HTTPS. Caddy handles Let's Encrypt issuance automatically, so the change is mostly a config exercise.

One brief operational note. During an early deploy session I accidentally pasted a GitHub personal-access token into the droplet's terminal scrollback. I rotated the token through the GitHub web interface within a couple of minutes and ran `history -c` to clear the local record. Nothing attributable to the token has shown up in the GitHub audit log since.

The lesson, which is to use deploy keys rather than PATs for unattended pulls, has been adopted.

CHAPTER 10: TESTING AND RESULTS

10.1 How I tested

My testing was pragmatic, not exhaustive. I cared more about end-to-end behaviour than unit coverage. The failure modes I worried about (a stale streak counter, an over-credited heart, a wrong leaderboard rank) are exactly the ones that only surface when you actually run the full flow, not when you mock the dependencies out.

The pure utility modules (`streak.util.ts`, `hearts.util.ts`, the reward calculator) got tested through small driver scripts that called them with handcrafted inputs and printed the results so I could eyeball them against the gamification spec in `Helper/gamification_spec.md`. Yes, this is informal. The surface is small enough that it worked. A proper Jest suite is on the future-work list.

10.2 Curl scripts

I exercised the HTTP surface with `curl` against both a local container and the production droplet. A small bash script walks through the lifecycle of a brand-new user.

1. `POST /api/auth/register` creates the user and returns a JWT.
2. `POST /api/me/enrollments/exam` enrolls them in JEE Main.
3. `GET /api/courses/exams` confirms the exam list comes back.
4. `GET /api/courses/exercises/<id>` pulls an exercise.
5. `POST /api/exercise/complete` gets called with `[true, true, ..., true]` ten times to submit a perfect attempt.
6. `GET /api/me/stats` confirms 25 XP got added (20 base + 5 perfect bonus), all five hearts are present, the streak is 1, and the user landed in a tier-1 leaderboard group.

The negative cases I exercised:

- A results array of the wrong length, returns 400.
- A submission with zero hearts (after deliberately depleting them first), returns 403 with the `OUT_OF_HEARTS` code.
- A request to `/me/stats` without an `Authorization` header, returns 401.
- A request to `/admin/subjects` with a wrong admin token, returns 401.

10.3 The phone

Day-to-day testing happened on a Samsung Galaxy A series device, with Expo Go for fast iteration and the production AAB sideloaded for the final validation rounds.

The single most valuable automated testing tool I used on this project was Google Play's pre-launch report. They run the AAB on a small fleet of physical Android devices in their lab and

produce a screenshot for each screen plus a stack trace for any crash. Both patches now living in `mobile/patches/` exist because of crashes the pre-launch report caught. Neither would have been caught locally. Both are Android 15-specific, and my test phone was on Android 14.

10.4 What actually shipped

At the time of writing, here's where things stand.

- A signed AAB is on the Google Play internal testing track. It installs cleanly, no warnings, on both a stock Pixel emulator and the physical test phone.
- The API is reachable at <http://144.126.255.209:3001/api> and returns live JSON for every documented endpoint.
- The seed pipeline produces 24 subjects, 124 units, 465 exercises, and 4,650 questions in around seven seconds.
- A new user can run the entire loop end to end (registration, enrolment, exercise completion, heart loss, XP gain, leaderboard placement) in under a minute on the 2GB droplet.
- The codebase is around 4,800 lines of TypeScript across the four sub-projects, plus the Prisma schema and the seed data.

So every committed objective from Section 1.4 is met. The platform is in a state where I can hand it to external testers.

A few qualitative things worth recording too. The celebration screen, the one that animates the rewards in sequence at the end of an exercise, ended up landing harder than the small amount of effort I spent on the animation timings would suggest. The unit feed (the zigzag layout of `LessonNode` instances) is the screen early users mention most often. It's also the screen where the visual debt to Duolingo is the most obvious. The leaderboard initially felt empty during testing because there was only one participant per group, but it got noticeably more fun once a few friends signed up and the groups started filling.

For the sake of completeness, a quick load test using `wrk` against `GET /api/courses/exams` on the production droplet held above 200 requests per second from a single client, with p95 latency under 50 ms. That's way above what the capstone needs. A more thorough stress test would be sensible before any production rollout.

10.5 Sample API calls

Production base URL: <http://144.126.255.209:3001/api>.

Register:

POST /api/auth/register
Content-Type: application/json

```
{
  "email": "demo@example.com",
  "password": "demo-1234",
  "displayName": "Demo",
  "timezone": "Asia/Kolkata"
}
```

Response carries the user summary plus a signed JWT.

Submit a completed exercise:

POST /api/exercise/complete
Authorization: Bearer <jwt>
Content-Type: application/json

```
{
  "exerciseId": "ckxyz...",
  "results": [true, true, false, true, true, true, true, true, true,
true]
}
```

Response:

```
{
  "scorePct": 90,
  "isPerfect": false,
  "xpAwarded": 20,
  "diamondsAwarded": 0,
  "heartsRemaining": 4,
  "streakOutcome": "CONTINUED",
  "currentStreak": 3,
  "longestStreak": 5,
  "unitCompleted": false
}
```

Fetch leaderboard:

```
GET /api/leaderboard/me
Authorization: Bearer <jwt>
```

Response carries the user's group, the user's rank, the twelve participants ordered by seasonXp, the league tier, and the seconds remaining in the season.

CHAPTER 11: WHAT'S BROKEN AND WHAT'S NEXT

11.1 Mistakes I made

A report that only writes up the wins is suspicious. The mistakes below are probably the most useful thing I'm taking away from this whole project. The lessons from them will outlast the codebase by a long way.

Starting on SQLite was a mistake. The first six weeks of development ran on it because I could commit the file straight to Git, which felt convenient at the time. Migrating to Postgres later was more painful than just starting on Postgres would have been. SQLite handles foreign keys differently, has slightly different constraint syntax, and doesn't support some of the index types I needed. I lost about half a day reconciling all of that. The lesson is to use the production database from day one, even if `docker compose up` is marginally heavier than a local `.db` file.

Modelling each question type as its own table was another one. It looked clean at first, every table only had the columns it actually needed, very tidy. Then I added the fifth question type and suddenly every controller, every service, every API response had to dispatch on the type, and the dispatch code was about a third of the question logic. Folding it all back into one table with a flat `content` JSON deleted that whole layer in one afternoon. The lesson is that flexibility at the edges of the schema usually beats structural neatness in the middle.

The first deploy was done by hand over SSH, command by command, with no script. The second deploy hit a problem I'd already solved on the first one, but with no script I tried to reapply the fix from memory and got it wrong. After about an hour of broken production I gave up, wrote `deploy.sh`, and every deploy after that went through it. The lesson is that even a five-line script remembers things better than I do.

Pasting a personal-access token into a terminal, which I already wrote up in Chapter 9. The lesson is to use deploy keys, not PATs, for unattended pulls.

Putting off Jest tests was probably the biggest one. Writing tests in the first few weeks felt like overhead I didn't really need. By the time I actually wanted tests, the surface had grown enough that going back to add them felt like a depressing chunk of work. So I haven't, yet.

The lesson here is that the per-test cost of writing tests is small if you spread it across the project, and it's brutal if you concentrate it at the end.

And I underestimated the Play Store. I assumed uploading the AAB was the last step. It turned out to be the start of a whole second workflow. Pre-launch report. Privacy policy. Data-safety form. Content rating. Screenshots. Feature graphic. Short description. Long description. The first submission ate most of a day. The lesson is that publishing to a store is its own project, not a single task.

11.2 Limitations

What the platform doesn't have, listed honestly. There's no TLS yet, the API is on plain HTTP, and putting Caddy in front of it is the next operational thing on the list. There are no push notifications either, so a streak that's about to break doesn't ping you, and a season ending doesn't ping you either. Those are well-known, high-leverage hooks in a Duolingo-style loop and not having them is definitely costing me retention. Infrastructure is single-region. Both the droplet and the Upstash instance are in BLR1, so anyone outside India sees worse latency than they need to. The corpus is small, 4,650 questions is fine for early testing but it isn't enough for serious exam prep. Analytics are thin, I record `ExerciseAttempt` rows but there isn't an aggregate dashboard for the learner or the operator yet. The tests are thin too, the pure utilities have driver-script coverage but there's no formal Jest suite. And there's no iOS build. The Expo project is iOS-buildable in principle, I just don't have an Apple developer account.

11.3 Future plans

The roadmap is loose, partly on purpose, partly because I've already learned that pinning it down too tightly mostly produces stale plans.

Reach is the obvious starting point. A domain, Caddy in front of the API for TLS, a push to the Play Store production track, and eventually an iOS build through EAS once I've sorted out an Apple developer account. None of those is interesting work. All of them have to happen.

Then there's the corpus. 4,650 questions is enough for early users to get a feel for the app and not enough for anyone actually preparing for an exam. I want to get to roughly 50,000 across the current five exams before I add a sixth. The sixth would probably be CAT, which means writing a verbal-reasoning question type, and I haven't fully thought through what that widget looks like yet. A spaced-repetition pass over `ExerciseAttempt` is also on the list — wrong answers should come back at growing intervals until they stop being wrong. And a streak-freeze item priced in diamonds, so somebody who has a genuinely impossible Tuesday doesn't lose a 90-day streak over it.

Engagement-wise, push notifications via Expo are the big one. Streak warnings, league results, the usual. Beyond that, friend-based leagues sitting next to the anonymous twelve-person ones, a small daily-quest system, and at some point a cosmetic shop where diamonds buy profile flair. None of that needs schema changes worth worrying about.

CHAPTER 12: CONCLUSION

The platform got built end to end during the capstone period and it's actually sitting on a public server right now, which still feels a bit unreal to type. Someone who installs the Android build can register, enrol, complete an exercise, gain XP, lose a heart, climb a tier, build up a daily streak, and then watch that streak break the next day if they slack off. The schema, the server, the phone app and the admin dashboard were all written by one person, in one repo, with mainstream open-source tooling. Honestly, that's also a useful reminder that the gap between "I have a project idea" and "the project is on the Play Store" is shorter than it tends to feel at the start. Not easy. Just shorter.

The big technical calls (Prisma over hand-written SQL, JSON-as-string over JSONB, JWT over session cookies, Expo over bare React Native, Docker Compose over a managed Kubernetes cluster, plain React contexts over a state library) each traded a bit of sophistication for a lot of reproducibility. That trade paid off most clearly during deployment, where I could move the whole system onto a fresh droplet inside one afternoon and have everything come back up the way it was.

The product is small, but it's internally honest, which I think matters more. The streak counter does what it claims. The leaderboard sorts on a real season-XP score. The heart timer ticks at the same rate on the server as it does on the client. The question bank is hand-written and big enough to support meaningful early use. And the mechanics that drive Duolingo's retention in language learning seem to translate cleanly to entrance-exam practice, which is the core bet behind the whole thing.

The remaining work is well scoped and incremental. TLS, push notifications, a wider corpus, an adaptive selector, an iOS build, friend-based leagues. Each of these is an addition to the current architecture, not a rewrite of it. Which means the project is in a pretty good position to keep going beyond the capstone milestone, if I want it to.

Three lessons I'm carrying out of this, in plain words. Pick your production database from day one. Write the deploy script before the second deploy, not after it. And ship the entire loop before polishing any single screen, because the loop is what makes the product worth using at all, and any single screen is just one part of it.

CHAPTER 13: REFERENCES

1. NestJS framework documentation. <https://docs.nestjs.com> (accessed April 2026).
2. Prisma documentation. <https://www.prisma.io/docs> (accessed April 2026).
3. PostgreSQL 16 documentation. <https://www.postgresql.org/docs/16/> (accessed April 2026).
4. Redis sorted set commands. <https://redis.io/docs/data-types/sorted-sets/> (accessed April 2026).
5. Expo SDK 51 release notes. <https://docs.expo.dev/versions/v51.0.0> (accessed April 2026).
6. React Native 0.74 documentation. <https://reactnative.dev/docs/0.74/getting-started> (accessed April 2026).
7. expo-router documentation. <https://docs.expo.dev/router/introduction/> (accessed April 2026).
8. Next.js 14 App Router documentation. <https://nextjs.org/docs/app> (accessed April 2026).
9. TailwindCSS documentation. <https://tailwindcss.com/docs> (accessed April 2026).
10. React Hook Form documentation. <https://react-hook-form.com/get-started> (accessed April 2026).
11. Zod schema validation. <https://zod.dev> (accessed April 2026).
12. Google Play target API level requirements. <https://developer.android.com/google/play/requirements/target-sdk> (accessed April 2026).
13. JSON Web Token specification (RFC 7519). <https://datatracker.ietf.org/doc/html/rfc7519>.
14. bcrypt password hashing. Provos, N. and Mazières, D. "A future-adaptable password scheme." USENIX Annual Technical Conference, 1999.
15. DigitalOcean Droplets documentation. <https://docs.digitalocean.com/products/droplets/> (accessed April 2026).
16. Upstash Redis documentation. <https://docs.upstash.com/redis> (accessed April 2026).
17. Docker Compose specification. <https://docs.docker.com/compose/compose-file/> (accessed April 2026).
18. Caddy server documentation. <https://caddyserver.com/docs/> (accessed April 2026).
19. patch-package. <https://github.com/ds300/patch-package> (accessed April 2026).
20. von Ahn, L. "Duolingo: Learn a Language for Free While Helping to Translate the Web." Talks and follow-up coverage on the role of gamification in language learning, 2013–2024.

End of report.

